1. Ana-(Pre)log

Why Analog Designers Need to Understand Fabrication Before Design

As analog designers, we're directly limited by what our fab can actually make. Unlike digital designers who work with pre-characterized standard cells, we're designing with individual transistors, resistors, and capacitors whose behavior depends entirely on the fabrication process.

Key reasons this matters:

1. Device matching and mismatch characteristics
2. Parasitic capacitances and resistances
3. Voltage limitations and breakdown voltages
4. Temperature coefficients and process variations

TinyTapeout: Your Cheap but Real Silicon

Here in UWASIC, we use an Educational Tapeout called TinyTapeout, which houses thousands of different projects on the same chip. TinyTapeout is how we learn analog design by making custom ASICs affordable. Instead of needing $100K+ for a chip, you can get your analog design fabricated for under $200 (Pre-Inflation).

How TinyTapeout Works

The sharing concept:

- One large chip is divided into hundreds of small "tiles"
- Each designer gets one or more tiles (160×100 µm each)
- All designs are fabricated together on the same chip
- You get a development board with YOUR design actually working in silicon
- Everyone pays a small fraction of the total fabrication cost

What you get:

- Your actual design fabricated in silicon
- Development board with RP2040 microcontroller
- Testing infrastructure and GPIO connections
- Access to a global community of chip designers

Silicon Chip Manufacturing Basics

Silicon chips start with ultra-pure silicon wafers - imagine a 12-inch diameter, paper-thin disc of crystalline silicon that's 99.9999999% pure. This silicon substrate provides the foundation for all our devices.

Key layers built on silicon:

1. Silicon substrate - The foundation
2. Isolation - Separates different circuit blocks
3. Active devices - Transistors, diodes
4. Local interconnect - Short connections
5. Metal layers - Routing and power distribution

SKY130 Process Technology

SKY130 is the world's first fully open-source Process Design Kit (PDK). Developed by Google and SkyWater Technology, it's a mature 130nm process that's perfect for learning analog design.

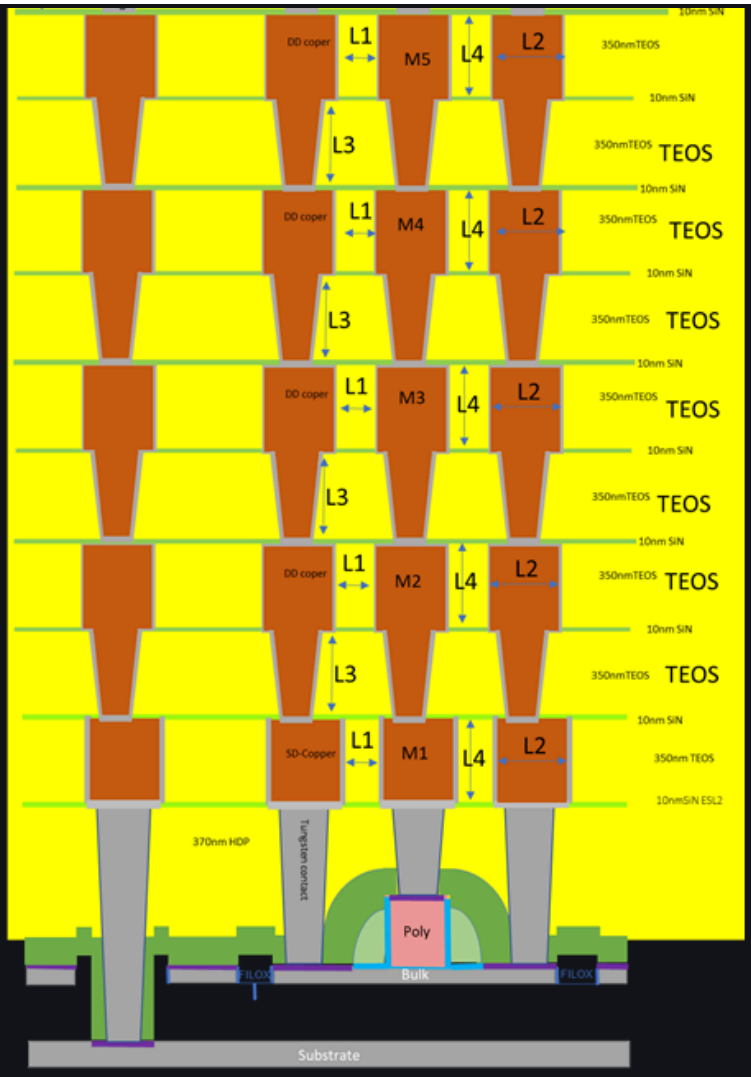
Available Metal Layers

SKY130 gives you 6 metal layers for routing:

- Local Interconnect (LI) - 12.8 Ω/□ - For very short connections

1. Metal 1 (M1) - 0.125 Ω/□ - Local routing, horizontal preferred
2. Metal 2 (M2) - 0.125 Ω/□ - Local routing, vertical preferred
3. Metal 3 (M3) - 0.047 Ω/□ - Intermediate routing
4. Metal 4 (M4) - 0.047 Ω/□ - Power distribution
5. Metal 5 (M5) - 0.0285 Ω/□ - Global routing, lowest resistance

For TinyTapeout analog designs, you can use ALL metal layers EXCEPT Metal 5!



5 layers of Copper Interconnect

Device Options for Analog Design

Transistors:

- 1.8V NMOS/PMOS - Your standard devices
- 3.3V Extended drain - For higher voltage interfaces

Resistors:

- P+ Poly resistors - 300 Ω/□, good matching
- P- Poly resistors - 2000 Ω/□, ultra-high resistance
- Metal resistors - Using metal layers

Capacitors:

- MiM capacitors - Metal-Insulator-Metal, 2.0 fF/μm²
- VPP capacitors - Vertical parallel plate
- MOS capacitors - Using transistor gates

Special devices:

- Varactors - Voltage-variable capacitors
- Bipolar transistors - NPN and PNP
- ESD protection diodes

TinyTapeout Analog Design Constraints

Design Area and Tiles

Analog designs require minimum 2 tiles high because analog circuits need more area and power supply complexity. Available configurations (out of many):

- 1×2 tiles - 160×200 μm (good for simple circuits)
- 2×2 tiles - 320×200 μm (better for complex analog)

Power Supply Options

- Standard 1.8V designs:
- VDPWR (1.8V digital power) VGND (ground)
- Mixed 1.8V/3.3V designs:
- VDPWR (1.8V digital power) VAPWR (3.3V analog power) VGND (ground)

Power routing rules:

- Power must be vertical stripes on Metal 4
- Must extend from bottom 10μm to top 10μm of your tile
- Minimum 1.2μm width

Analog Pin Specifications

Available analog pins: ua[0] through ua[5] (maximum 6 pins) Must use sequentially starting from ua[0] Pricing:

- First 4 analog pins: $40 each
- Additional pins: $100 each
- Plus $50 per tile for area

Important Design Rules

- You CANNOT use Metal 5 - It's reserved for TinyTapeout's power grid
- Pin locations fixed - Must use official TinyTapeout templates
- Unused digital outputs - Must connect to ground (don't leave floating) Use templates exactly - Available on GitHub

---

Now that we've covered how the fabrication works, it is time to learn about the specifics of the op-amp we are building. To get started, head to the 2. A Basic Operational Amplifier section, enough yap, lets get something working!

2. The Op-Amp

Your Challenge

Design Description:

Finally the moment you've been waiting for - what is your task? Your task is to build an Operational Amplifier with 2 Stages just so we can have differential input, single ended output. The gain wouldn't be as great as we normally want it, however the challenge comes in the Design Specifications. You are free to choose BJT or MosFETs, but make sure to research the benefits of each.

Design Specifications:

- Minimum differential gain: 20 dB (10x voltage gain)
- Maximum input offset voltage: 5 mV
- Minimum CMRR: 40 dB
- Input impedance: > 1 MΩ
- Output impedance: < 1 kΩ
- Power consumption: < 5 mW (with ±1.8V supplies)

This single-stage design will help you understand:

- How differential pairs work
- The trade-offs between gain and complexity
- The fundamental building blocks of more complex op amps.

Not familiar with OpAmps? Read Below

In this page, you will be introduced to the component referred to as the Operational Amplifier, and

the differences between your course knowledge's ideal circumstances, versus reality which introduces lots of factors that make the component non-ideal. If you are a first year, this would be a good time to cover Operational Amplifier in your book before reading this.

Op Amps: Ideal vs Reality

Sedra Microelectronics, Page 60
From a signal point of view the op amp has three terminals: two differential input terminals and one output terminal, amplifiers require connections to 2 dc power to operate

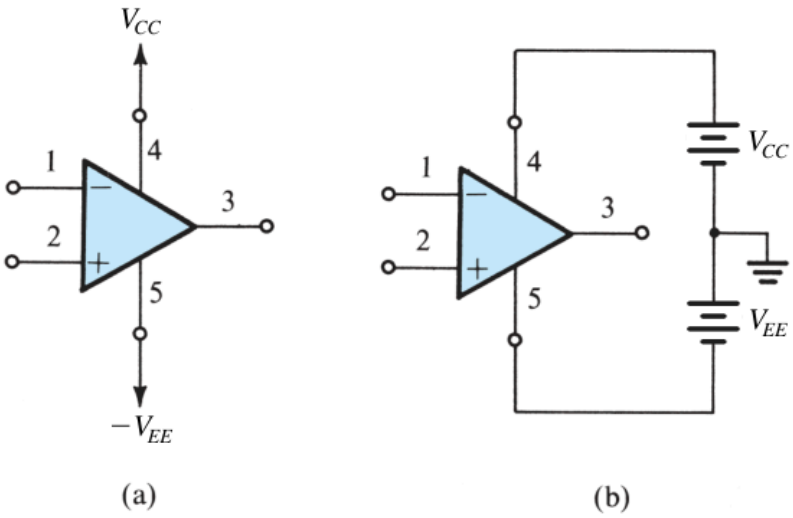


Figure 2.2 The op amp shown connected to dc power supplies.

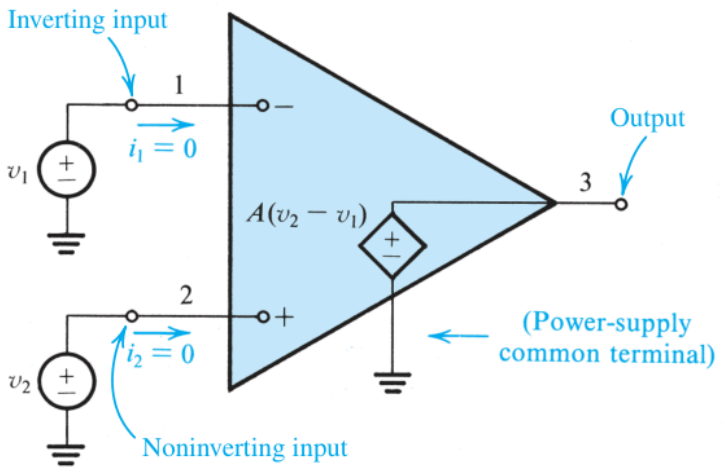


Figure 2.3 Equivalent circuit of the ideal op amp.

Terminal Definitions:

- V+: Non-inverting input
- V-: Inverting input
- Vout: Output voltage
- A: Open-loop gain (differential gain)

Fundamental Relationship: Vout = A(V+ - V-)

Ideal Properties:

- Infinite input impedance (no current flows into inputs)
- Zero output impedance (can drive any load)
- Infinite open-loop gain ($A \rightarrow \infty$)
- Infinite bandwidth (gain constant at all frequencies)
- Perfect common-mode rejection (only responds to differential signals)
- Zero offset voltage (Vout = 0 when V+ = V-)

Reality Check

The reality is that, the component uses to make up these operational amplifiers themselves are non-linear, hence the Vout would never be linear itself, here are a few real properties:

- Finite input impedance (typically 10^6 to 10^{12} Ω)
- Non-zero output impedance (typically 10-100 Ω)
- Limited open-loop gain (typically 10^4 to 10^6)
- Finite bandwidth (gain rolls off with frequency)
- Finite common-mode rejection ratio (CMRR ~60-120 dB)
- Input offset voltage (typically 1-10 mV)

- Input bias currents (pA to nA range)
- Slew rate limitations
- Power supply limitations

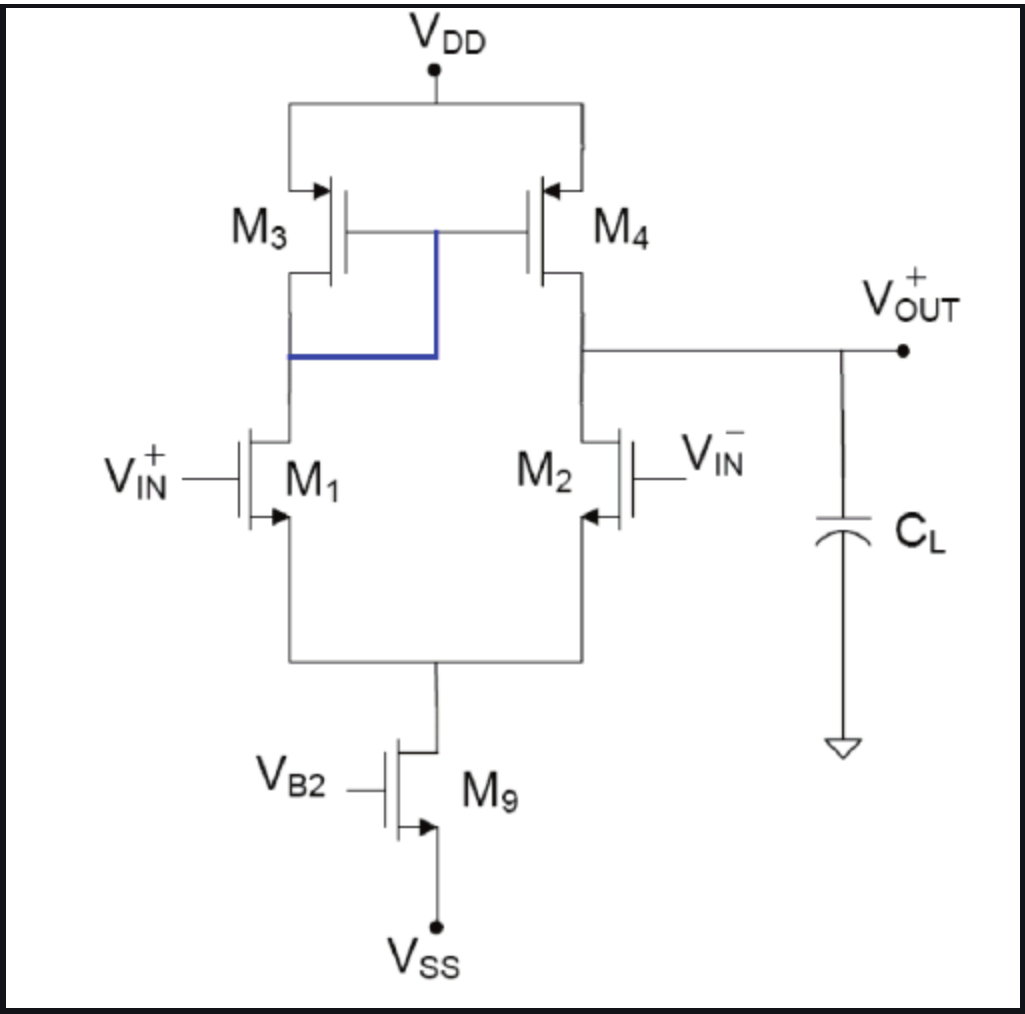
Internal Structure of Op-Amps

Normally Operational Amplifiers consist of 2 stages, where there is a 3rd stage however it is optional and depends on the usage of the op amp. In each of these stages, you will use a lot of transistors, and this might be a good area to learn about transistors. For now, we will only cover MOSFETs.

Additional Reading: MOSFET Reference

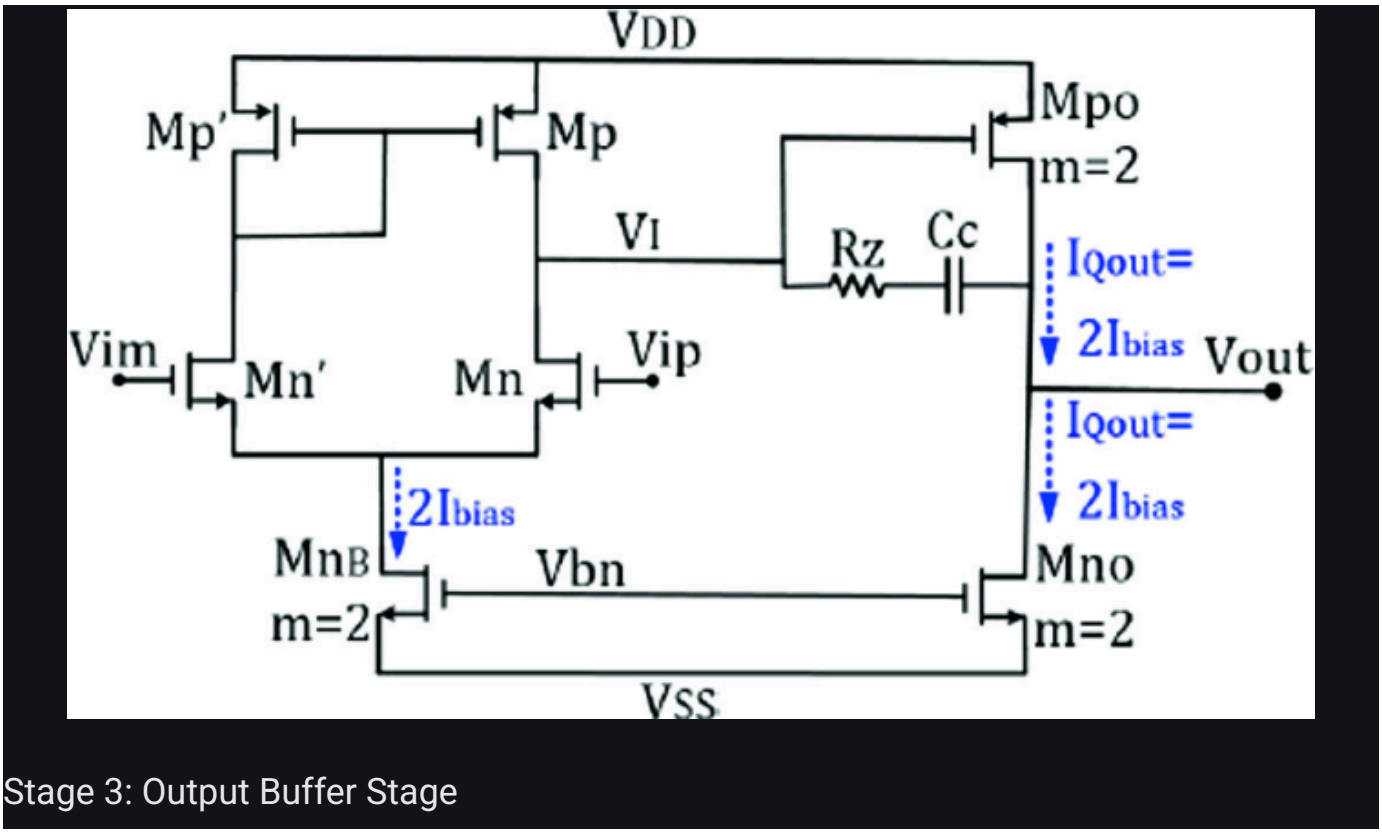
Stage 1: Differential Input Stage

- Function: Converts the differential input voltage (V+ - V-) into a differential current. Responsible for the Differential and High Input Impedance Propety.
- Typical Implementation: Differential pair (BJT or MOSFET)
- Key Characteristics:
  - High input impedance (Needed for Ideal Property of having 0 input current)
  - Good common-mode rejection (Ignores common signals, like an Op Amp)
  - Converts voltage difference to current difference



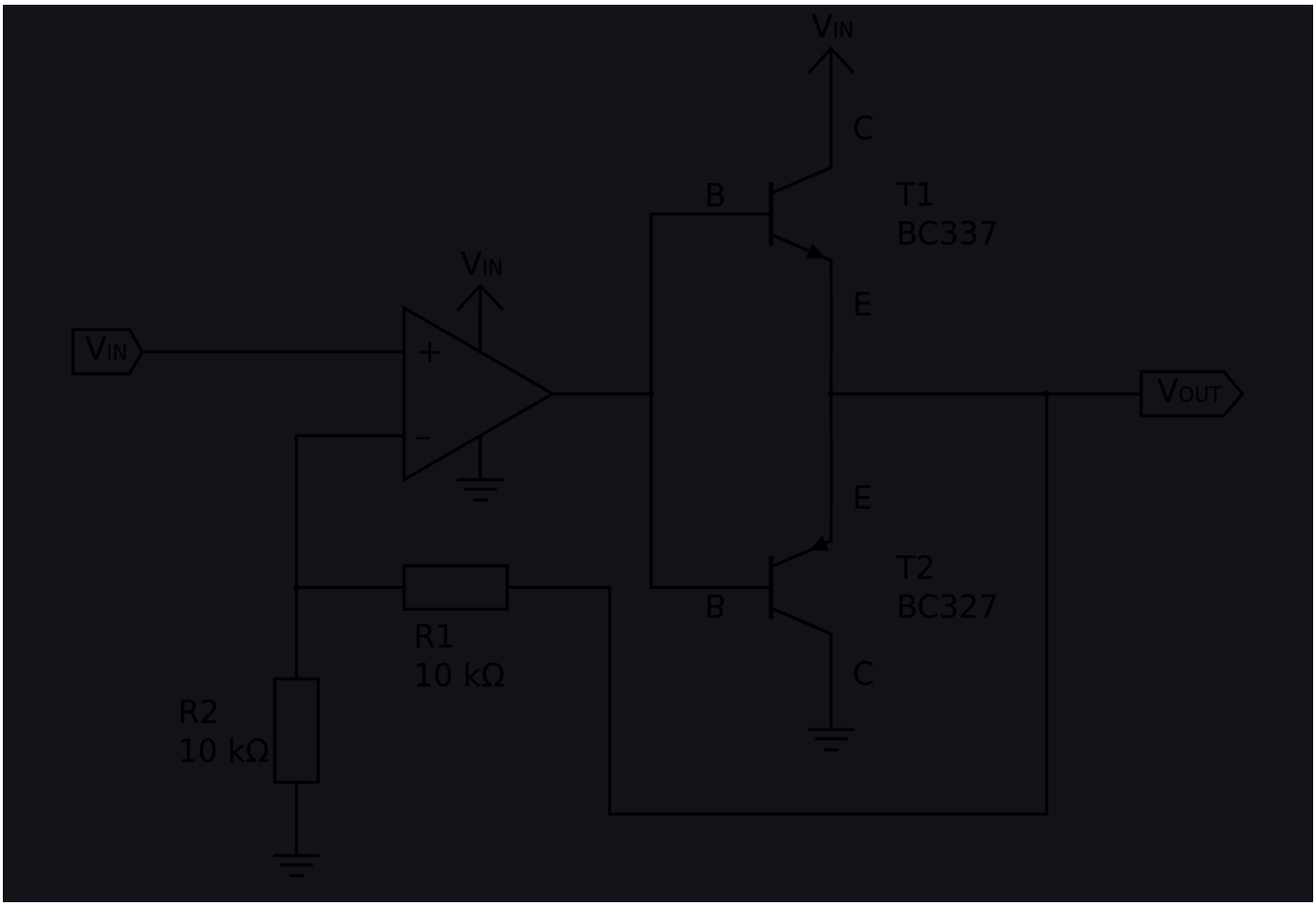
Stage 2: Gain Stage (Voltage Amplification)

- Function: Responsible for the Differential/Open-Loop Gain Constant.
- Typical Implementation: High-gain amplifier (often using common-emitter/source stages)
- Key Characteristics:
  - Very high voltage gain (60-100 dB)
  - Often includes frequency compensation
  - May have multiple cascaded gain stages



Stage 3: Output Buffer Stage

- Function: Provides current drive capability and low output impedance
- Typical Implementation: Push-pull output stage (Class AB)
- Key Characteristics:
  - Low output impedance
  - High current drive capability
  - Rail-to-rail or near rail-to-rail output swing



Take the left hand side amplifier as a 2-stage amplifier, the RHS of the circuit is the buffer stage

All these stages add up to the properties we want for an Operational Amplifier:

Differential Stage:

- High Input Impedance
  - → Negligible Input Current

Gain Stage:

- High Differential/Open-Loop Gain

Output Buffer Stage:

- Low Output Impedance

However, a weakness still remains which is the low bandwidth due to the capacitance associated with transistors. Read more about Miller Capacitance.

Now that you have received your task, head to 3. Intro to Tools to understand which tools you will be using to implement the following.

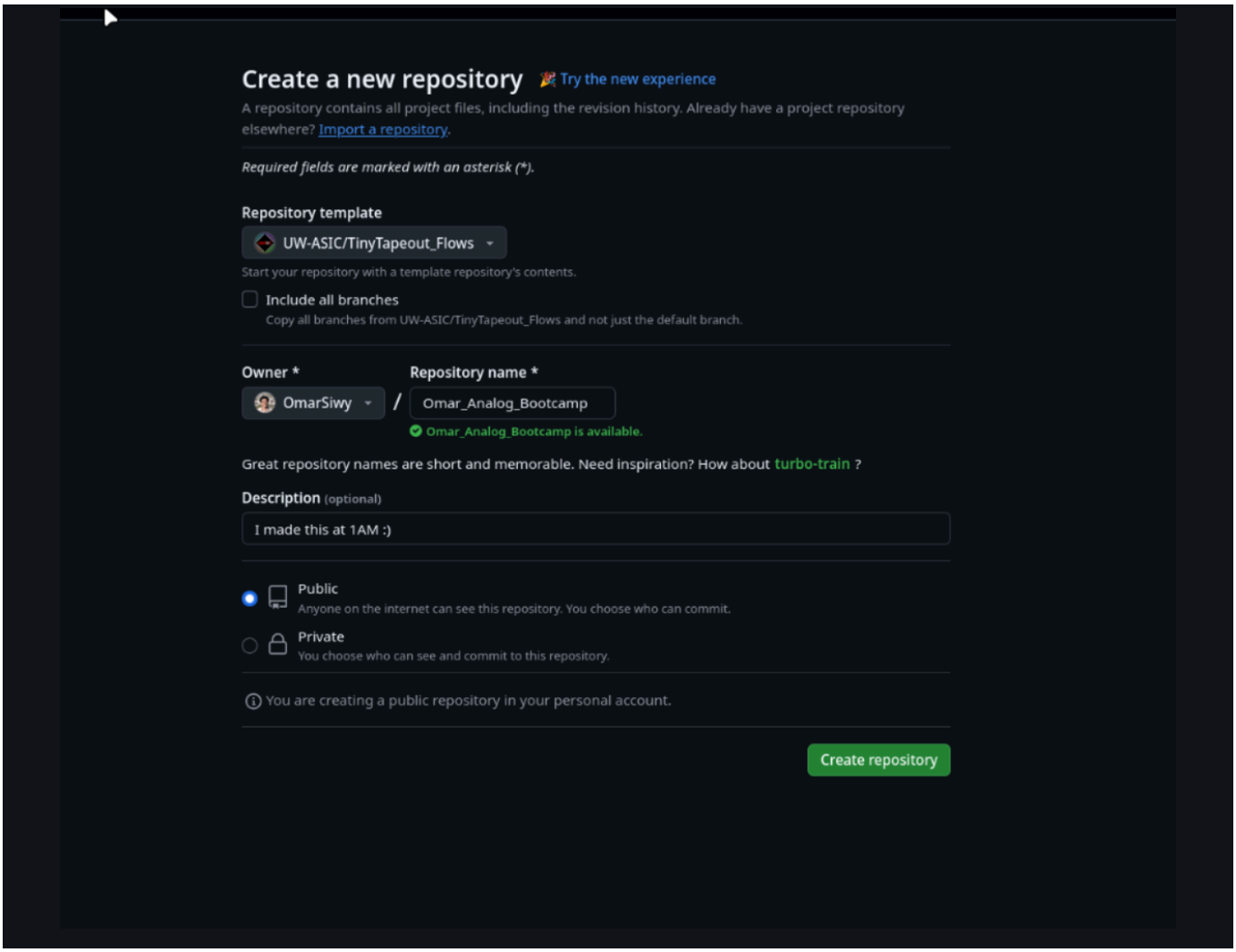
3. Setting up Tooling

Due to the financial limitations, all of our tooling is derived from Open-Source tools that replicate what the industry tooling does, and even sometimes with more features than the industry.

Project Setup

Step 1: Create Your Repository

1. Go to https://github.com/UW-ASIC/TinyTapeout_Flows
2. Click "Use this template" → "Create a new repository"
3. Name your repository:
4. `[your-discord_username]_Analog_Bootcamp`
5. Make it Public
6. Click "Create repository"



example

Step 2: Clone Your New Repository

- if you are using windows, please install WSL (Windows subsystem for linux) first, then git clone it inside the subsystem.

```
git clone
https://github.com/[your-username]/[your-username]-bootcamp-analog-project.git
```

```
cd [your-username]-bootcamp-analog-project
```

1. What is Nix?

Nix is your "software time machine" - it ensures everyone on your team uses the exact same tool versions, regardless of their operating system.

Why Nix Matters for Hardware Teams

Problem: "It works on my machine!"

- Person A has Magic 8.3.60, Person B has Magic 8.3.100
- Different ngspice versions give different simulation results
- Linux vs macOS vs Windows compatibility issues
- Spending hours debugging environment instead of designing circuits

Nix solution: Everyone gets identical tools

- Same Magic version, same ngspice, same everything
- Works identically on Linux, macOS, and Windows (via WSL2)
- No more "Did you install the dependencies?" conversations

2. Using Nix as your default installer for everything

Installation (One-time setup)

Linux/WSL2:

```
# Install Nix
sh <(curl -L https://nixos.org/nix/install) --no-daemon

# Enable modern Nix features
mkdir -p ~/.config/nix
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf

# Restart your shell
exec $SHELL
```

macOS:

```
# Due to finicky mac support,
# all mac users must run a docker wrapper around nix-shell
# Which makes it slower, but useable!
chmod +x mac_shell.sh

./mac_shell.sh
```

Windows:

```
# In PowerShell as Admin
wsl --install

# Restart, then in WSL2 Ubuntu:
sh <(curl -L https://nixos.org/nix/install) --no-daemon
```

```
# Enable modern Nix features
mkdir -p ~/.config/nix
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf

# Restart your shell
exec $SHELL
```

To enter the Nix Environment (Daily Usage):

```
# Start your day
cd <PROJECT_FOLDER>
./env.sh # Make sure the shell.nix file is in the same directory

# Exit when done
exit
```

3. Create your analog project through the flow

Step 1: Enter Nix Environment

```
cd [your-username]_Analog_Bootcamp
# For WSL and Linux users
./env.sh

# NOTE: Whenever you get python installation errors of UWASIC-ALG or gmid, then
you need to,
rm -rf .venv

# Then re-run the shell
```

Step 2: Create Project

```
# Navigate to flows directory
cd flows/

# Create your analog project
make CreateProject PROJECT_NAME=bootcamp_opamp PROJECT_TYPE=analog

# Check that it worked
make status

# Now leave the flows/ because it is only used to create the project, not for
actual calls

cd ..
```

You should see:

```
Current project state: analog
Projects created:
  bootcamp_opamp (analog)
```

```
...And a bunch of other things
```

Step 3: Understand your Project Structure:

```
your-project/
├── analog/
│   └── bootcamp_opamp/
│       ├── build/              # Build system and tools
│       │   ├── schematic/      # XSchem workflow
│       │   ├── layout/         # Magic/Klayout workflow
│       │   └── validation/     # DRC/LVS verification
│       ├── schematics/         # Your .sch and .sym files go here
│       │   ├── testbenches/    # Schematic's testbenches
│       ├── library/            # Our Shared Library for shared components
```

```
│       └── layout/             # Your .mag layout files
```

At this point, we will teach you how to use the software, but this project is your little baby, so make sure you take care of it.

Now Head to 4. Schematic Design For a user guide on how to use XSchem for the schematics you will create.

4. Schematic Design

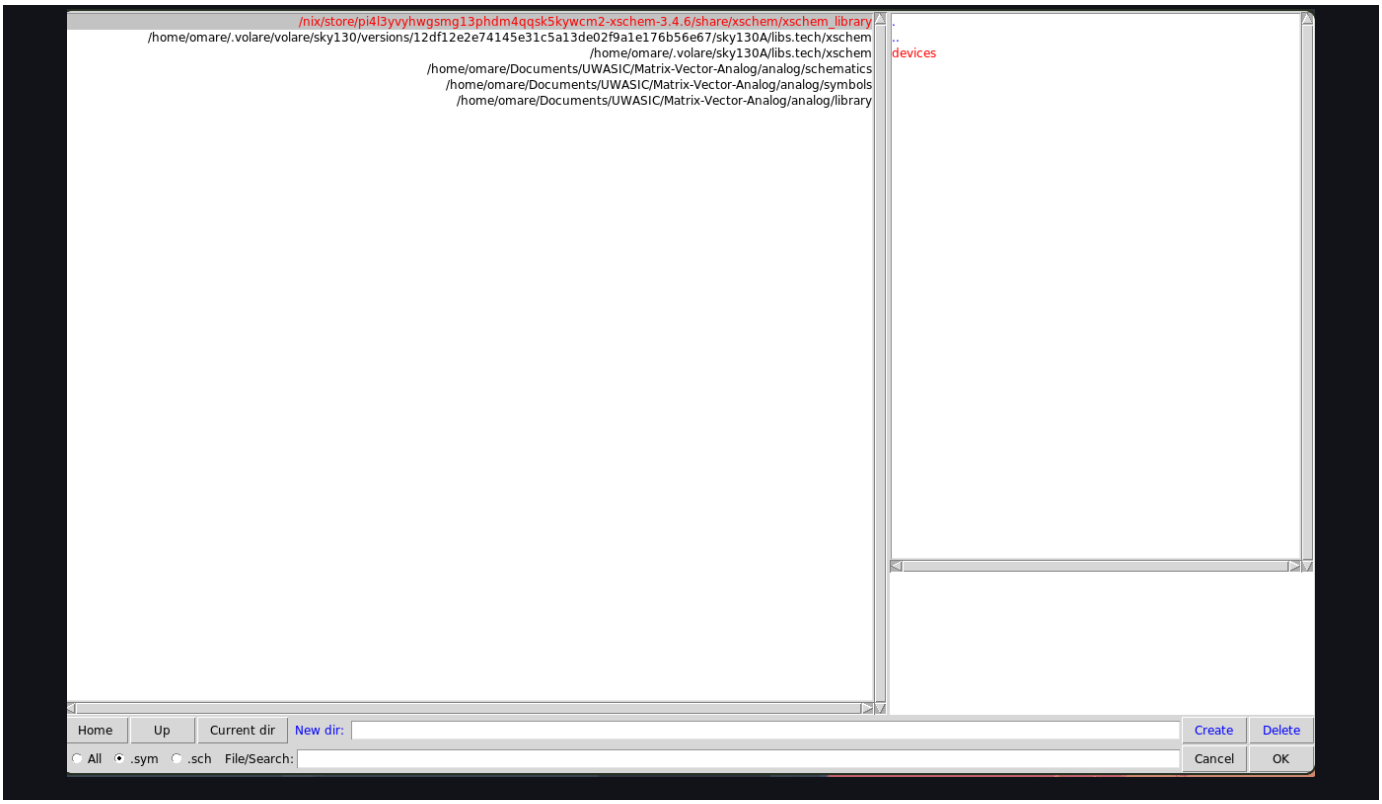
Time to bring your op-amp from theory to reality! In this section, you'll learn to use XSchem to create a schematic of your single-stage operational amplifier.

Starting XSchem for Your Project

```
# Navigate to your schematic build directory
cd analog/build/schematic

# Launch XSchem with proper SKY130 setup
make schematic
```

Access to Components



Component Selection Menu by pressing Shift + i

In your component selection, you will see several folders. You are only allowed to use:

xschem_library:

- Contains all the devices you need for simulations and circuit labeling
- These components will NOT be included in layout
  - DO NOT USE THESE IN THE SCHEMATIC

xschem/sky130_fd_pr:

- Contains the physical primitive devices from the SkyWater 130nm PDK that will be used in actual layout
- Includes MOSFETs (NMOS/PMOS) and Resistors and Capacitors:
  - 1.8V devices: sky130_fd_pr__nfet_01v8, sky130_fd_pr__pfet_01v8 (standard) [Use this]
  - 5V devices: sky130_fd_pr__nfet_g5v0d10v5, sky130_fd_pr__pfet_g5v0d10v5
- Passive components like resistors, capacitors, and diodes
- ESD protection devices

Essential XSchem Keybinds

Navigation:

- Mouse wheel - Zoom in/out
- F - Fit schematic to window
- Z - Zoom to selected area (click and drag)
- Shift + Arrow keys - Pan view

Component Placement:

- Insert - Insert component (opens library browser)
- W - Place wire (click-to-click routing)
- L - Place label/net name
- T - Place text annotation
- Escape - Cancel current operation

Selection and Editing:

- S - Select mode (click and drag to select area)
- M - Move selected objects
- C - Copy selected objects
- Delete - Delete selected objects
- R - Rotate selected objects 90°
- Shift+R - Rotate selected objects -90°

Important Operations:

- Q - Edit properties of selected object
- Ctrl+S - Save schematic
- Ctrl+Z - Undos
- Ctrl+Y - Redo

Common Questions

1. How to Create New Schematic?

1. File → New Schematic
2. Save as
3. `opamp_two_stage.sch`

2. How to Get/Place Components?

1. Press Insert → Navigate to
2. `sky130_fd_pr/`
3. → Select Your choice of transistor

Note: Simulation components are always in *devices/*

Now Head to 5. Simulations & Verification For a user guide on how to test your Operational Amplifier, note you have a few design requirement so make sure you test everything.

5. Simulations & Verification

Now that you have your op-amp schematic complete, it's time to verify it meets all design specifications through comprehensive simulation. This page will guide you through setting up testbenches and analyzing your circuit's performance.

Recall Your Design Specifications

Your two-stage op-amp must meet these requirements:

- Minimum differential gain: 20 dB (10x voltage gain)
- Maximum input offset voltage: 5 mV
- Minimum CMRR: 40 dB
- Input impedance: > 1 MΩ
- Output impedance: < 1 kΩ
- Power consumption: < 5 mW (with 1.8V supply)

Setting Up Your Simulation Environment

Navigate to Simulation Directory

```
cd analog/bootcamp_opamp/build/schematic
```

```
make schematic  # Opens XSchem
```

Simulation Workflow Overview

Method 1: Integrated XSchem Simulation (Recommended for beginners)

1. Open your `opamp_two_stage.sch` in XSchem
2. Create a symbol from your schematic (File → Make Symbol)
3. Create new testbench: `opamp_two_stage_tb.sch`
4. Add your op-amp symbol to the testbench
5. Add simulation components and run directly from XSchem GUI

Method 2: Separate Testbench Files (Advanced)

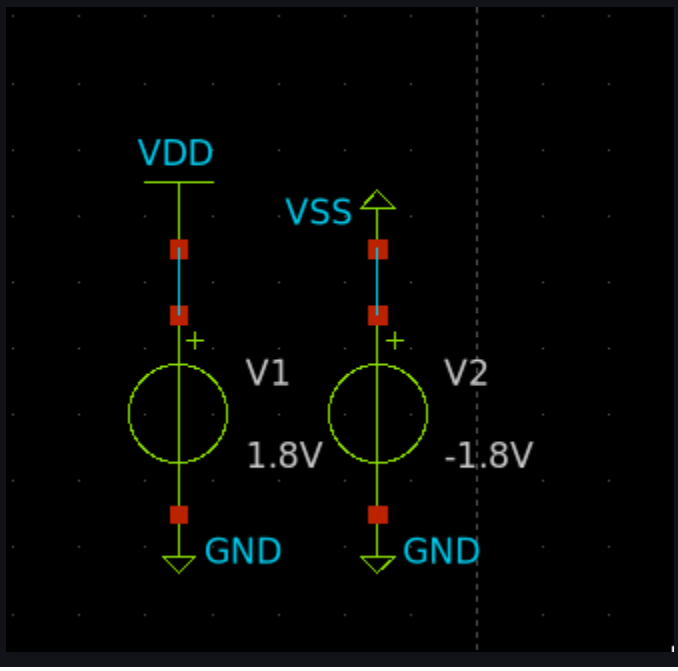
1. Create dedicated testbench schematics for each test
2. Instantiate your op-amp symbol in each testbench
3. Run batch simulations from command line using ngspice

```
# Generate netlist only
xschem -q -n tb_dc_analysis.sch

# Run in NGSpice
ngspice tb_dc_analysis.spice
```

Note: XSchem has built-in global VDD and VSS/(anything you rename it to)

You can use the VDD and VSS symbols from the library as long as you set them in your testbench to something use GND as a reference, GND is also a global component meaning its value is passed down to all components that use it, but it is always 0V, so we use it as reference like below:



Picture of setting VDD and VSS in testbench for opamp to use it internally without needing to have an explicit pin for it.

Creating Your Symbol

Before creating testbenches, you need a symbol for your op-amp:

Step 1: Generate Symbol from Schematic

1. With your `opamp_single_stage.sch` open, go to File → Make Symbol

2. XSchem automatically creates `opamp_single_stage.sym`
3. Edit the symbol if needed to make it look like a proper op-amp triangle
4. Ensure all pins are properly labeled: `VIN_P` `VIN_N` `VOUT` `VDD` `VSS`

Creating Comprehensive Testbenches

Testbench 1: DC Operating Point and Gain

Create the Testbench

1. File → New Schematic
2. Save as: `tb_dc_analysis.sch`

Add Components

1. Insert your op-amp symbol:
   - Press Insert → Navigate to current directory
   - Select `opamp_two_stage.sym`
2. Add power supplies:
   - Insert → `devices/vsource.sym`
   - VDD supply: Set properties `value=1.8 name=VDD`
   - VSS supply: Set properties `value=0 name=VSS`
   - Connect VDD to op-amp VDD pin, VSS to VSS pin and ground
3. Add input bias:
   - Insert → `devices/vsource.sym`
   - Common-mode bias: `value=0.9 name=VCM` (connects to both inputs through resistors)
   - Differential input: `value=0 name=VDIFF` (for DC sweep)
4. Add input resistors (for proper biasing):
   - Insert → `devices/res.sym`
   - Two 1MΩ resistors from VCM to each input
   - This sets the common-mode operating point
5. Add simulation directives:
   - Insert → `devices/code_shown.sym`
   - Insert → `sky130_fd_pr/corner.sym` required for importing library for transistor simulation

Testbench 2: AC Frequency Response

Create AC Testbench

1. File → New Schematic
2. Save as: `tb_ac_analysis.sch`
3. Copy the DC testbench and modify for AC analysis

Modify for AC Analysis

1. Change differential input source properties:
   - `value="dc 0 ac 1"` (DC=0V, AC=1V amplitude)

Testbench 3: Common-Mode Rejection Ratio (CMRR)

Create CMRR Testbench

1. File → New Schematic
2. Save as: `tb_cmrr.sch`
3. You should be able to get the CMRR based on one of the previous setups 😀

Testbench 4: Input and Output Impedance

Create Impedance Testbench

1. File → New Schematic
2. Save as: `tb_impedance.sch`
3. Add a test current source to measure input impedance

Performance Summary & Iteration

Create Your Results Table in the README.md

Fill in your measured results:

```
2-Stage Op-Amp Performance Summary
==================================
DC Gain:                _____ dB   (Target: ≥20 dB)
Input Offset:           _____ mV   (Target: ≤5 mV)
CMRR:                   _____ dB   (Target: ≥40 dB)
Input Impedance:        _____ MΩ   (Target: ≥1 MΩ)
Output Impedance:       _____ kΩ   (Target: ≤1 kΩ)
Power Consumption:      _____ mW   (Target: ≤5 mW)
3dB Bandwidth:          _____ MHz

GBW Product:            _____ MHz

PASS/FAIL: ___________
```

A worthy contender, Your circuit is ready for 6. Physical Layout and DRC/LVS Verification

Remember: Layout will affect performance, so having good margin on specifications is crucial for final success.

6. Physical Layout and DRC/LVS Verification

Congratulations on getting your op-amp simulations working! Now comes the critical step of transforming your schematic into a physical layout that can actually be fabricated. This is where analog design gets challenging - layout directly affects performance.

Why Layout Matters for Analog Circuits

Unlike digital circuits, analog layout directly impacts performance:

- Matching: Transistor mismatch creates offset voltage and degrades CMRR
- Parasitics: Layout adds unwanted capacitances and resistances
- Noise Coupling: Poor layout can introduce crosstalk and substrate noise
- Thermal Gradients: Device placement affects temperature matching

Starting Magic for Your Project

```
# Navigate to layout build directory
cd analog/bootcamp_opamp/build/layout

# Launch Magic with SKY130 technology file
make layout
```

This launches Magic with:

- SKY130 design rules loaded
- Proper layer stack configuration
- DRC engine enabled
- Technology-specific device generators
- The definition you picked out (the template)

Essential Magic Commands & Navigation

Basic Navigation

- Z - Zoom in (on cursor location)
- Shift+Z - Zoom out
- V - Fit layout to window
- Mouse wheel - Zoom
- Arrow keys - Pan view

Selection Operations

- S - Select area (click and drag box)
- A - Select all
- Shift+S - Add to selection
- Ctrl+D - Deselect all

Layer Operations

- P - Paint current layer
- E - Erase current layer
- Space - Toggle paint/erase mode

Metal Layer Shortcuts

- 1 - Metal 1 (m1) - horizontal local routing
- 2 - Metal 2 (m2) - vertical local routing
- 3 - Metal 3 (m3) - intermediate routing
- 4 - Metal 4 (m4) - power distribution
- Shift+1 - Via 1 (connects m1↔m2)
- Shift+2 - Via 2 (connects m2↔m3)
- Shift+3 - Via 3 (connects m3↔m4)

Important Magic Commands (Type in console)

- `:save` - Save layout
- `:load filename` - Load layout file
- `:drc check` - Run design rule check
- `:drc why` - Explain DRC error under cursor
- `:extract all` - Extract parasitics
- `:ext2spice lvs` - Generate LVS netlist

Design Rule Check (DRC)

Running DRC

```
# In Magic command line:
:drc check

# View DRC errors
```

```
:drc why
```

Common DRC Violations

Metal Spacing Violations:

- Fix: Increase spacing between metal traces
- Rule: Metal1 spacing ≥ 0.14μm

Via Enclosure Violations:

- Fix: Ensure vias are properly enclosed by metal
- Rule: Via must be enclosed by ≥0.055μm metal

Well Spacing Violations:

- Fix: Ensure n-well spacing ≥ 1.27μm
- Check p-well to n-well spacing

Device Rule Violations:

- Minimum transistor width: 0.42μm
- Minimum transistor length: 0.15μm
- Gate extension over active: ≥0.18μm

Layout vs. Schematic (LVS) Verification

Extracting Layout Netlist

```
# In Magic:
:extract all
:ext2spice lvs
:ext2spice cthresh 0.01
:ext2spice rthresh 0.01
```

```
:ext2spice
```

This creates `opamp_single_stage.spice` with extracted parasitics.

Running LVS Comparison

```
# Using netgen (if available)
netgen -batch lvs "opamp_single_stage.spice opamp_single_stage"
"schematic.spice opamp_single_stage"

# Manual comparison
# Compare device counts, connectivity, and parameter values
```

Common LVS Issues

Device Count Mismatch:

- Check for missing or extra devices in layout
- Verify multiplicity factors match

Connectivity Errors:

- Trace unconnected nets in layout
- Check for missing vias or broken connections
- Verify power supply connections

Parameter Mismatches:

- Check W/L values match between schematic and layout
- Verify nf and mult parameters

Post-Layout Simulation

Extract Parasitics

The extracted netlist includes:

- Resistance: Metal trace resistance
- Capacitance: Interconnect and junction capacitances
- Layout-dependent effects: Device matching, stress effects

Compare Performance

Run the same testbenches on extracted netlist:

```
# Simulate extracted layout
ngspice opamp_single_stage_extracted.spice

# Compare with schematic results:
# - DC gain (may be slightly lower)
# - Bandwidth (will be lower due to parasitics)
# - Offset voltage (may increase due to mismatch)
```

```
# - Power consumption (should be similar)
```

Layout Quality Metrics

Matching Quality

- Differential pair: Offset voltage < 2mV indicates good matching
- Current mirror: Output current mismatch < 5%

Parasitic Impact

- Gain degradation: Should be < 20%
- Bandwidth reduction: Acceptable if still meets specs
- Phase margin: Should remain > 45° for stability

Area Efficiency

- Total area: Minimize while meeting performance specs
- Aspect ratio: Keep reasonable for easy integration

Congratulations!

If you've reached here. You've completed the full analog IC design flow:

- ✅ Schematic Design - Created circuit topology
- ✅ Simulation - Verified performance
- ✅ Layout - Physical implementation
- ✅ Verification - DRC/LVS clean
- ✅ Post-layout Simulation - Confirmed final performance

Your single-stage operational amplifier is now ready for fabrication! This design flow forms the foundation for all analog IC design projects.

Next Steps: Tag @omarsawy or @elius for a design review by linking your github repository, then explore the advanced topics in 7. Self-Study while waiting for feedback.